TP4 AA1

Escriba aquí su nombre completo

Indicaciones básicas

0. Debe usar este notebook como template para su entrega. Haga una copia y comience a completar las consignas.
1. Cada uno debe completar las consignas indicadas en este notebook.
2.
3. No pueden repetir el mismo dataset que ya haya definido un compañero.
4. copias explícitas de secciones enteras del trabajo de otro será penalizado disminuyendo su puntuación.
5. No se olvide de añadir las fuentes de inspiración de su código (blogs, prompts de chatgpt o similar).
6. Además de todo el código que agregue, es importante que sepa interpretarlo. Agregue texto explicativo en cada sección. Esto le ayudará al momento del coloquio / parcial
7. Revise las fecha límite de entrega de este trabajo

ENTREGA

- En el foro destinado a la tarea debe postear al menos tres veces:
  - La primera vez para indicar el dataset elegido
  - Una segunda vez para postear su entrega
  - Una tercera vez para brindar feedback a alguien más en el foro.
  - Puede comentar más de un posteo de sus compañeros siempre y cuando el intercambio sea respetuoso y fructífero.
- Deben postear como solución un enlace a su notebook colab público y también un enlace al mismo notebook alojado en su repositorio GitHub.
- Debe sumar el enlace a un video donde muestre su solución y explique lo realizado. Duración máxima: 5 - 10 minutos. Puede grabarse a Ud. mismo usando una sesión de zoom y grabar localmente. Puede subir el video a su youtube personal como privado o Unlisted.
- Enlaces no accesibles o enlaces erróneos es igual a determinar que la tarea no ha sido entregada. Corrobore la viabilidad de los enlaces que postea.

Tarea: Aplicación de Regresión Lineal y Análisis de Importancia de Variables

Objetivo: Aplicar un modelo de regresión lineal a un dataset de su elección, evaluar su rendimiento e identificar las variables más relevantes para la predicción.

Instrucciones:

1. Selección del Dataset

- Elijan un dataset de UCI ML Repository del siguiente enlace: https://archive.ics.uci.edu/datasets/?Task=Regression&skip=0&take=10&sort=desc&orderBy=NumHits&search=
- Requisitos:
  - Debe tener al menos 4 variables numéricas continuas (1 target, 3 o más features).
  - Idealmente, que las features tengan distintas escalas o unidades (no excluyente).
  - Revisar en el foro de la tarea que dicho dataset no haya sido ya elegido por otra persona.
  - Postee en el foro de la tarea el dataset que eligió. Continue al siguiente punto.

Resolución:

Haz doble clic (o ingresa) para editar

```
!pip install ucimlrepo
```

```
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from ucimlrepo) (2.2.3)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.13/dist-packages (from ucimlrepo) (2026.7.22)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.0.0->ucimlrepo) (2026.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: ucimlrepo
Successfully installed ucimlrepo-0.0.7
```

```
from ucimlrepo import fetch_ucirepo


air_quality = fetch_ucirepo(id=360)

X = air_quality.data.features # Las variables (sensores, temperatura, humedad)

print(X.head())
```

```
         Date      Time  CO(GT)  PT08.S1(CO)  NMHC(GT)  C6H6(GT)  \
0  3/10/2004  18:00:00     2.6         1360       150      11.9
1  3/10/2004  19:00:00     2.0         1292       112       9.4
2  3/10/2004  20:00:00     2.2         1402        88       9.0
3  3/10/2004  21:00:00     2.2         1376        80       9.2
4  3/10/2004  22:00:00     1.6         1272        51       6.5

   PT08.S2(NMHC)  NOx(GT)  PT08.S3(NOx)  NO2(GT)  PT08.S4(NO2)  PT08.S5(O3)  \
0           1046      166          1056      113          1692         1268
1            955      103          1174       92          1559          972
2            939      131          1140      114          1555         1074
3            948      172          1092      122          1584         1203
4            836      131          1205      116          1490         1110

      T    RH      AH
0  13.6  48.9  0.7578
1  13.3  47.7  0.7255
2  11.9  54.0  0.7502
3  11.0  60.0  0.7867
4  11.2  59.6  0.7888
```

```
import pandas as pd
import numpy as np

```

```
df = pd.DataFrame(X)
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	3/10/2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578
1	3/10/2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255
2	3/10/2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502
3	3/10/2004	21:00:00	2.2	1376	80	9.2	948	172	1092	122	1584	1203	11.0	60.0	0.7867
4	3/10/2004	22:00:00	1.6	1272	51	6.5	836	131	1205	116	1490	1110	11.2	59.6	0.7888

Variable	Descripción
Date	Fecha de la medición.
Time	Hora de la medición.
CO(GT)	Concentración real de **Monóxido de Carbono (CO, Carbon Monoxide)** medida por un equipo de referencia. **GT (Ground Truth)** indica que es una medición certificada considerada como valor real.
PT08.S1(CO)	Lectura del sensor químico **PT08 Sensor 1**, diseñado para detectar variaciones asociadas al **Monóxido de Carbono (CO)**.
NMHC(GT)	Concentración real de **Hidrocarburos No Metánicos (NMHC, Non-Methane HydroCarbons)** medida por un equipo de referencia.
C6H6(GT)	Concentración real de **Benceno (C_6H_6)**, un compuesto orgánico volátil presente en combustibles y emisiones vehiculares.
PT08.S2(NMHC)	Lectura del sensor químico **PT08 Sensor 2**, asociado a la detección de **Hidrocarburos No Metánicos (NMHC)**.
NOx(GT)	Concentración real de **Óxidos de Nitrógeno (NOx, Nitrogen Oxides)**, grupo de gases contaminantes formados principalmente por NO y NO_2.
PT08.S3(NOx)	Lectura del sensor químico **PT08 Sensor 3**, asociado a la detección de **Óxidos de Nitrógeno (NOx)**.
NO2(GT)	Concentración real de **Dióxido de Nitrógeno (NO_2, Nitrogen Dioxide)** medida por un equipo de referencia.
PT08.S4(NO2)	Lectura del sensor químico **PT08 Sensor 4**, asociado a la detección de **Dióxido de Nitrógeno (NO_2)**.
PT08.S5(O3)	Lectura del sensor químico **PT08 Sensor 5**, relacionado con la detección de **Ozono (O_3, Ozone)**.
T	**Temperatura (Temperature)** del aire expresada en grados Celsius (°C).
RH	**Humedad Relativa (Relative Humidity)** expresada como porcentaje (%). Indica qué tan cerca está el aire de alcanzar la saturación de vapor de agua.
AH	**Humedad Absoluta (Absolute Humidity)**. Representa la cantidad real de vapor de agua presente en el aire.

DataSet utilizado : Air Quality (UCI)

Es un dataSet publicado originalmente publicado en 2008 posteriormente incorporado en los repositorios de la UCI (universidad de California en Irvine).

- este dataset contiene datos de mediciones de sensores que fueron instalados en una ciudad italiana en zona urbana con alta contaminación y a nivel de calle.

Los datos de este conjunto de datos fueron recolectados con dos tipos de sensores :

- Equipos profesionales certificados ⟶ miden la concentración real del contaminante. Sensores químicos de bajo costo (PT08) ⟶ intentan estimar esa concentración.
- Sensores químicos de bajo costo (PT08) ⟶ intentan estimar esa concentración.

Variables medidas con sensores de bajo costo

Variable	Sensor asociado
PT08.S1(CO)	Sensor sensible al CO
PT08.S2(NMHC)	Sensor sensible a hidrocarburos
PT08.S3(NOx)	Sensor sensible a NOx
PT08.S4(NO2)	Sensor sensible a NO_2
PT08.S5(O3)	Sensor relacionado con O_3

Características:

✅ Mucho más baratos.

✅ Fáciles de instalar.

✅ Consumen poca energía.

❌ Menos precisos.

❌ Se ven afectados por temperatura y humedad.

❌ Su precisión puede degradarse con el tiempo (sensor drift).

Variables medidas con equipos certificados (Ground Truth)

Variable	Qué mide
CO(GT)	Concentración real de monóxido de carbono
NMHC(GT)	Concentración real de hidrocarburos no metánicos
C6H6(GT)	Concentración real de benceno
NOx(GT)	Concentración real de óxidos de nitrógeno
NO2(GT)	Concentración real de dióxido de nitrógeno

Características:

✅ Muy precisos.

✅ Están calibrados bajo normas internacionales.

✅ Pueden medir directamente la concentración real del contaminante.

❌ Son caros.

❌ Requieren mantenimiento especializado.

❌ Consumen más energía.

❌ Suelen requerir personal especializado.

❌ Es difícil instalar una gran cantidad de ellos en una ciudad.

El objetivo de este análisis es evaluar si las mediciones obtenidas por sensores de bajo costo, junto con variables meteorológicas como la temperatura y la humedad, permiten estimar con precisión las mediciones realizadas por equipos certificados de referencia.

La variable objetivo seleccionada es CO(GT), que representa la concentración de Monóxido de Carbono (CO) medida por un equipo certificado de alta precisión (Ground Truth).

La elección de esta variable se justifica por la importancia del monóxido de carbono como contaminante atmosférico y su impacto sobre la salud humana. Según el Ministerio de Salud de Argentina, la exposición a altas concentraciones de monóxido de carbono reduce la capacidad de la sangre para transportar oxígeno, impidiendo que las células utilicen adecuadamente el oxígeno disponible. Esta disminución del aporte de oxígeno afecta principalmente al cerebro y al corazón, pudiendo provocar síntomas que van desde dolores de cabeza y mareos hasta consecuencias graves en exposiciones prolongadas o a concentraciones elevadas.

Por lo tanto, se buscará desarrollar un modelo predictivo capaz de estimar la concentración de CO(GT) a partir de las lecturas de los sensores PT08 (sensores mas accesibles y menos precisos) y de las variables meteorológicas disponibles en el dataset.

2. **Análisis exploratorio (previo al modelado)**

- Describan las variables (media, distribución, outliers).
- Visualizen:
  - Histogramas o boxplots para ver distribuciones.
  - Gráficos de dispersión (scatterplots) entre features y target.
- **Pregunta clave**: ¿Qué relaciones lineales preliminares observan?

```
1 df.head(3)
```

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	3/10/2004	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578
1	3/10/2004	19:00:00	2.0	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255
2	3/10/2004	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502

```
1 display (df.columns)
2 df.info()
```

```
Index(['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)',
       'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)',
       'PT08.S5(O3)', 'T', 'RH', 'AH'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9357 entries, 0 to 9356
Data columns (total 15 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Date           9357 non-null   object
 1   Time           9357 non-null   object
 2   CO(GT)         9357 non-null   float64
 3   PT08.S1(CO)    9357 non-null   int64
 4   NMHC(GT)       9357 non-null   int64
 5   C6H6(GT)       9357 non-null   float64
 6   PT08.S2(NMHC)  9357 non-null   int64
 7   NOx(GT)        9357 non-null   int64
 8   PT08.S3(NOx)   9357 non-null   int64
 9   NO2(GT)        9357 non-null   int64
 10  PT08.S4(NO2)   9357 non-null   int64
 11  PT08.S5(O3)    9357 non-null   int64
 12  T              9357 non-null   float64
 13  RH             9357 non-null   float64
 14  AH             9357 non-null   float64
dtypes: float64(5), int64(8), object(2)
memory usage: 1.1+ MB
```

```
1 # convierto las variables object a formato fecha o hora
2 df['Date'] = pd.to_datetime(df['Date']).dt.date
3 df['Time'] = pd.to_datetime(df['Time']).dt.time
```

```
/tmp/ipykernel_1633/2341628233.py:3: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please speci
  df['Time'] = pd.to_datetime(df['Time']).dt.time
```

```
1 display(df['Date'].min(),df['Date'].max())
```

```
datetime.date(2004, 3, 10)
datetime.date(2005, 4, 4)
```

- El conjunto de datos no presenta valores nulos en ninguna de sus variables
- se presentan 2 variables de tipo fecha y hora (cada registro representa las mediciones de cada hora de cada dia desde el 10/03/2004 hasta el 04/04/2005)
- se presentan 5 mediciones de tipo numerico flotante
- Se presentan 8 mediciones de tipo entero

```
1 df = df[['CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)',
2        'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)',
3        'PT08.S5(O3)', 'T', 'RH', 'AH']]
4
5 df.describe()
```

	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
count	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000	9357.000000
mean	-34.207524	1048.990061	-159.090093	1.865683	894.595276	168.616971	794.990168	58.148873	1391.479641	975.072032	9.778305	39.485380	-6.837604
std	77.657170	329.832710	139.789093	41.380206	342.333252	257.433866	321.993552	126.940455	467.210125	456.938184	43.203623	51.216145	38.976670
min	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000	-200.000000
25%	0.600000	921.000000	-200.000000	4.000000	711.000000	50.000000	637.000000	53.000000	1185.000000	700.000000	10.900000	34.100000	0.692300
50%	1.500000	1053.000000	-200.000000	7.900000	895.000000	141.000000	794.000000	96.000000	1446.000000	942.000000	17.200000	48.600000	0.976800
75%	2.600000	1221.000000	-200.000000	13.600000	1105.000000	284.000000	960.000000	133.000000	1662.000000	1255.000000	24.100000	61.900000	1.296200
max	11.900000	2040.000000	1189.000000	63.700000	2214.000000	1479.000000	2683.000000	340.000000	2775.000000	2523.000000	44.600000	88.700000	2.231000

- Se puede observar como todas las variables tienen sus mediciones en distintas escalas y manejan distintos grados de variación entre las mismas, pero hay una extraña coincidencia en los minimos de todas las variables (todas las variables tienen un minimo en -200)
- -200 según la documentación representa la no medición o la falta de datos.

```
1 df = df.replace(-200,np.nan)
2 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9357 entries, 0 to 9356
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CO(GT)         7674 non-null   float64
 1   PT08.S1(CO)    8991 non-null   float64
 2   NMHC(GT)       914 non-null    float64
 3   C6H6(GT)       8991 non-null   float64
 4   PT08.S2(NMHC)  8991 non-null   float64
 5   NOx(GT)        7718 non-null   float64
 6   PT08.S3(NOx)   8991 non-null   float64
 7   NO2(GT)        7715 non-null   float64
 8   PT08.S4(NO2)   8991 non-null   float64
 9   PT08.S5(O3)    8991 non-null   float64
 10  T              8991 non-null   float64
 11  RH             8991 non-null   float64
 12  AH             8991 non-null   float64
dtypes: float64(13)
memory usage: 950.4 KB
```

```
1 display(f'NO2(GT) : tiene {df['NO2(GT)'].isna().sum()} nulos ({(df['NO2(GT)'].isna().sum() / df.shape[0]*100).round(2)}%), NOx(GT) : tiene {df['NOx(GT)'].isna().sum()} nulos ({(df['NOx(GT)'
```

```
'NO2(GT) : tiene 1642 nulos (17.55%), NOx(GT) : tiene 1639 nulos (17.52%),  NMHC(GT) : tiene 8443 nulos (90.23%), CO(GT) : tiene 1683 nulos (17.99%)'
```

NMHC(GT) : es una variable muy poco confiable a la hora de analizar o utilizarla para entrenar un modelo ya que el mas del 90% de sus datos son nulos

ahora detecto

```
import matplotlib.pyplot as plt
import seaborn as sns


dark = {
    "axes.facecolor": "#202123",
    "figure.facecolor": "#202123",
    "axes.edgecolor": "#CCCCCC",
    "axes.labelcolor": "#E0E0E0",
    "xtick.color": "#E0E0E0",
    "ytick.color": "#E0E0E0",
    "grid.color": "#444444",
    "grid.linestyle": "--",
    "grid.alpha": 0.5,
    "text.color": "#E0E0E0",
    "lines.color": "#10A37F",
    "patch.edgecolor": "#E0E0E0",
    "axes.titleweight": "bold",
    "axes.titlesize": 20,
    "axes.labelsize": 18
}

sns.set_theme(style=dark)
colores = sns.color_palette("Set2", n_colors=len(df.columns))

titulos = [
    'CO(GT) --- (Monóxido de Carbono Real)',
    'PT08.S1(CO) --- (Sensor de Monóxido de Carbono)',
    'NMHC(GT) --- (Hidrocarburos No Metánicos Reales)',
    'C6H6(GT) --- (Benceno Real)',
    'PT08.S2(NMHC) --- (Sensor de Hidrocarburos No Metánicos)',
    'NOx(GT) --- (Óxidos de Nitrógeno Reales)',
    'PT08.S3(NOx) --- (Sensor de Óxidos de Nitrógeno)',
    'NO2(GT) --- (Dióxido de Nitrógeno Real)',
    'PT08.S4(NO2) --- (Sensor de Dióxido de Nitrógeno)',
    'PT08.S5(O3) --- (Sensor de Ozono)',
    'T --- (Temperatura)',
    'RH --- (Humedad Relativa)',
    'AH --- (Humedad Absoluta)'
]

# 1. Definir columnas de la cuadrícula y calcular las filas automáticamente
n_cols_grid = 3
n_filas_grid = int(np.ceil(len(df.columns) / n_cols_grid))

# 2. Crear la figura dinámica
figbox, axesbox = plt.subplots(n_filas_grid, n_cols_grid, figsize=(20, n_filas_grid * 3.5))

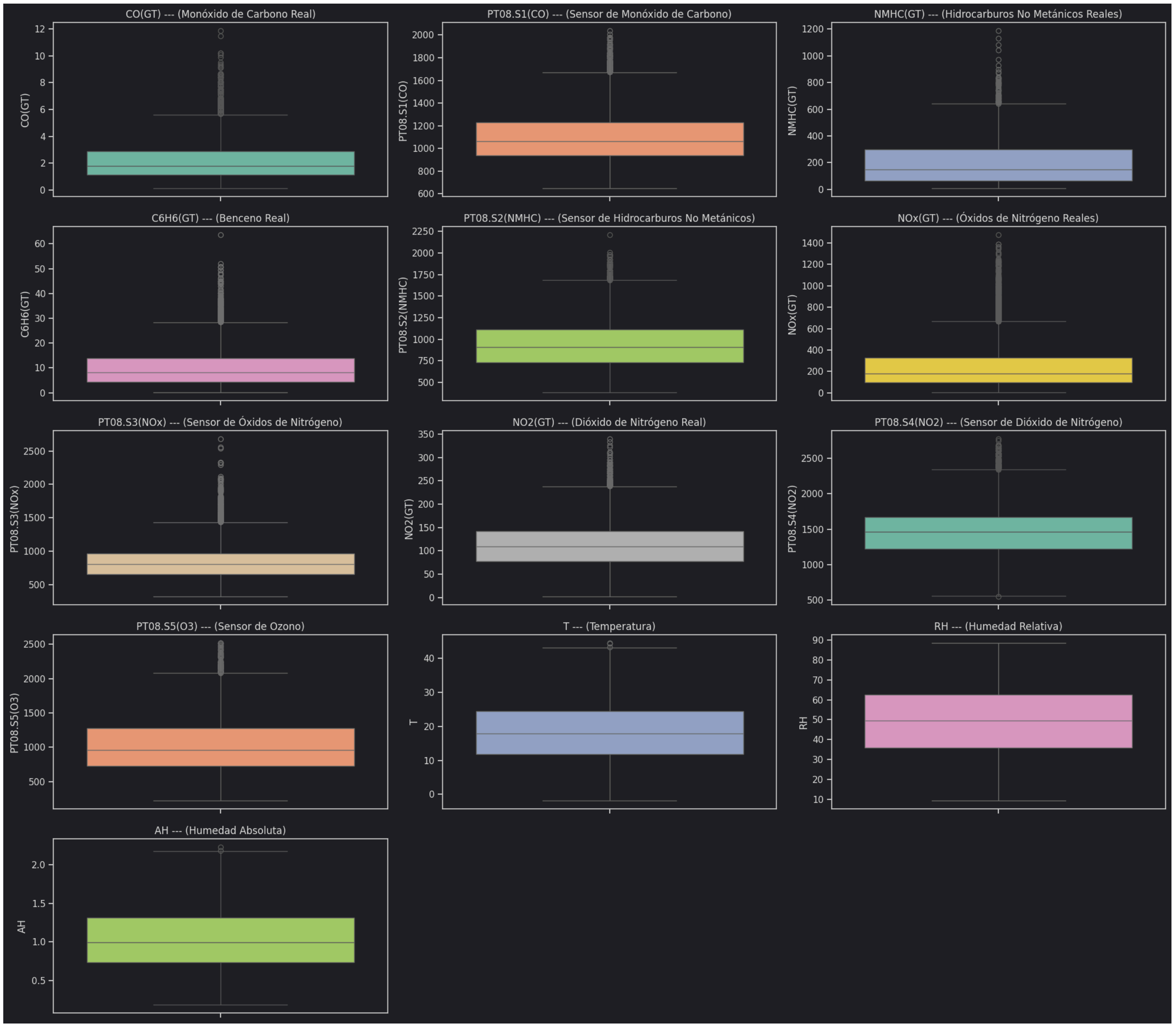
# 3. Aplanar la matriz de ejes para usar un solo índice simple [i]
axes_flat = axesbox.flatten()

# 4. Graficar cada columna
for i, column in enumerate(df.columns):
    sns.boxplot(data=df, y=column, ax=axes_flat[i],color=colores[i])
    axes_flat[i].set_title(titulos[i])

# 5. Ocultar los ejes que queden vacíos si el total de columnas no es múltiplo de 3
for j in range(i + 1, len(axes_flat)):
    axes_flat[j].axis('off')

plt.tight_layout()
plt.show()

```



Las mediciones de Monóxido de Carbono (CO), Benceno (C_6H_6), Óxidos de Nitrógeno (NOx), Dióxido de Nitrógeno (NO_2) y las lecturas del sensor de Ozono (O_3) presentan una baja dispersión en el 50% central de los datos, evidenciada por un rango intercuartílico relativamente pequeño. Esto indica que los niveles habituales de estas variables se mantienen relativamente estables. Sin embargo, los boxplots muestran numerosos valores atípicos en el extremo superior, lo que sugiere la ocurrencia de episodios puntuales de contaminación significativamente más elevados que los niveles normales registrados durante el período de estudio.

Resolución:

Haz doble clic (o ingresa) para editar

```
import matplotlib.pyplot as plt
import seaborn as sns

colores = sns.color_palette("Set2", n_colors=len(df.columns))



# 1. Definir columnas de la cuadrícula y calcular las filas automáticamente
n_cols_grid = 3
n_filas_grid = int(np.ceil(len(df.columns) / n_cols_grid))

# 2. Crear la figura dinámica
figbox, axesbox = plt.subplots(n_filas_grid, n_cols_grid, figsize=(20, n_filas_grid * 3.5))

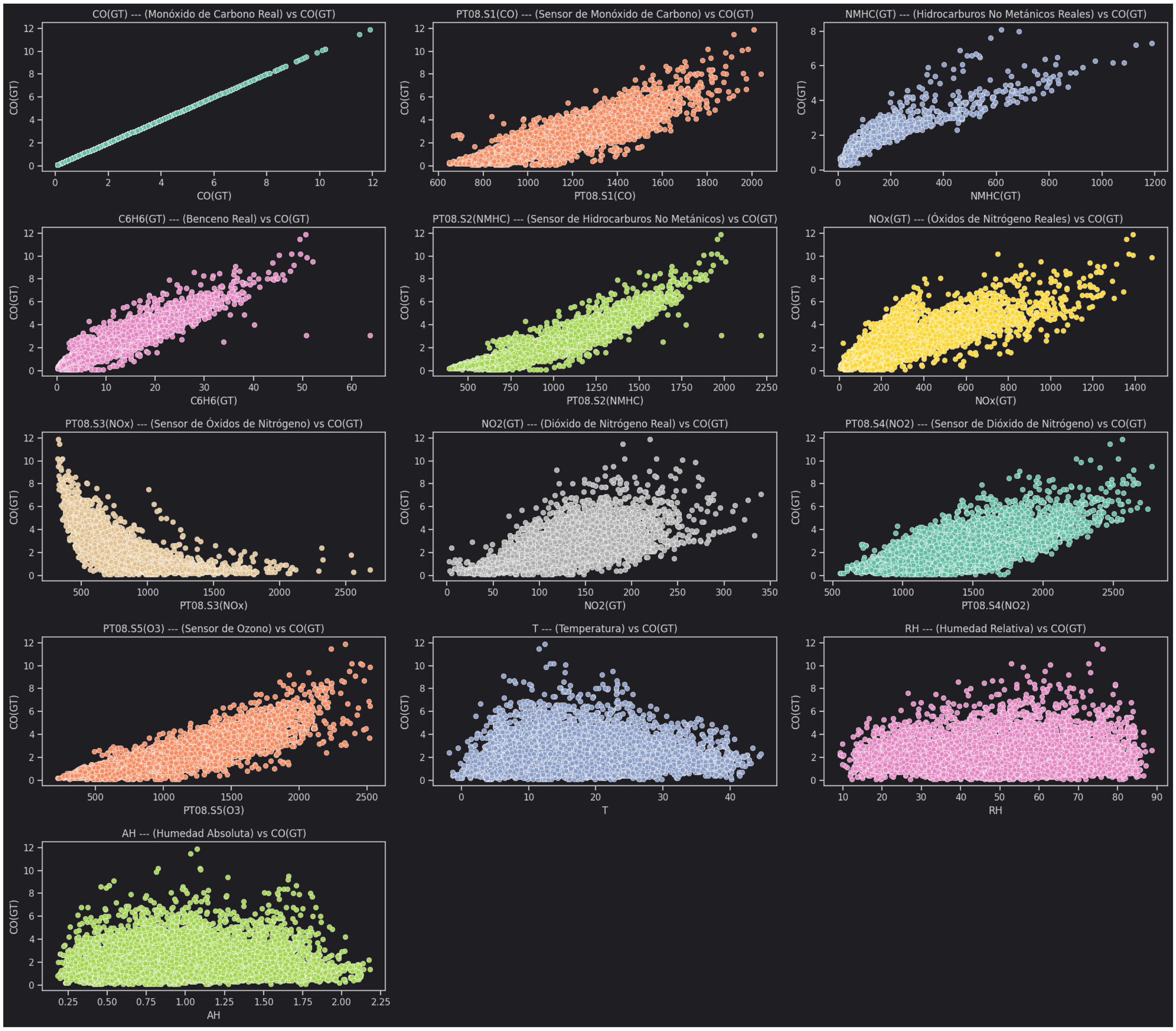
# 3. Aplanar la matriz de ejes para usar un solo índice simple [i]
axes_flat = axesbox.flatten()

# 4. Graficar cada columna
for i, column in enumerate(df):
        sns.scatterplot(data=df,x=column,y='CO(GT)', ax=axes_flat[i],color=colores[i])
        axes_flat[i].set_title(f'{titulos[i]} vs CO(GT)')

# 5. Ocultar los ejes que queden vacíos si el total de columnas no es múltiplo de 3
for j in range(i + 1, len(axes_flat)):
    axes_flat[j].axis('off')

plt.tight_layout()
plt.show()

```



- Visualmente puedo notar que existen relaciones fuertemente positivas entre la variable objetivo (Concentración Real de Monóxido de Carbono) y las variables Lectura del Sensor de Monóxido de Carbono, Lectura del Sensor de Hidrocarburos No Metánicos, Concentración Real de Dióxido de Nitrógeno y Lectura del Sensor de Ozono.
- También detecto una posible logaritmica negativa entre la variable objetivo (Concentración Real de Monóxido de Carbono) y la Lectura del Sensor de Óxidos de Nitrógeno.
- Visualmente, no encuentro una relación significativa entre la variable objetivo (Concentración Real de Monóxido de Carbono) y las mediciones de Temperatura del Aire, Humedad Relativa y Humedad Absoluta.

```
from scipy.stats import pearsonr
from scipy.stats import spearmanr

list_pearson_spearman = []

for column in df:
    df_temp = df.loc[~df['CO(GT)'].isna(),:]
    if column != 'CO(GT)':
        col_temp = df_temp.loc[~df[column].isna(),:]
        pearson, _ = pearsonr(col_temp[column], col_temp['CO(GT)'])
        spearman, _ = spearmanr(col_temp[column], col_temp['CO(GT)'])
        list_pearson_spearman.append({'columna' : column ,'valor pearson' : pearson,'valor spearman' : spearman})

list_pearson_spearman = pd.DataFrame(list_pearson_spearman)
list_pearson_spearman.sort_values(by='valor pearson',ascending=False)
display(list_pearson_spearman)
```

	columna	valor pearson	valor spearman
0	PT08.S1(CO)	0.879288	0.879422
1	NMHC(GT)	0.889734	0.936076
2	C6H6(GT)	0.931078	0.926981
3	PT08.S2(NMHC)	0.915514	0.926978
4	NOx(GT)	0.795028	0.784544
5	PT08.S3(NOx)	-0.703446	-0.811156
6	NO2(GT)	0.683343	0.734537
7	PT08.S4(NO2)	0.630703	0.594367
8	PT08.S5(O3)	0.854182	0.855705
9	T	0.022109	0.068907
10	RH	0.048890	-0.006484
11	AH	0.048556	0.053475

Para el entrenamiento del modelo se utilizará como variable objetivo únicamente CO(GT), correspondiente a la medición de referencia de alta precisión de la concentración real de monóxido de carbono.

Como variables predictoras se emplearán todas las lecturas provenientes de los sensores de bajo costo (PT08.S1(CO), PT08.S2(NMHC), PT08.S3(NOx), PT08.S4(NO2) y PT08.S5(O3)), junto con las variables meteorológicas disponibles en el dataset: temperatura del aire, humedad relativa y humedad absoluta.

Las restantes mediciones de referencia (NMHC(GT), C6H6(GT), NOx(GT) y NO2(GT)) no serán utilizadas como variables predictoras, ya que el objetivo del estudio es evaluar la capacidad de los sensores de bajo costo y de las variables ambientales para estimar la concentración real de monóxido de carbono medida por un equipo certificado.

3. **Preprocesamiento**

- Limpieza: Manejen missing values (eliminar, imputar) y outliers (si es necesario).
- Limpieza: indique cuáles features descarta. Justifique.
- Indique si usará o no variables categóricas. Justifique. Realice su preprocesamiento adeucado.
- Escalen las features (p.ej., StandardScaler) para comparar coeficientes después.
- Dividan en train/test (70-30 o 80-20).

```
# voy a entrenar el modelo con los datos de los sensores quimicos de bajo costo así que excluyo a los sensores de referencia

X = df.loc[~df['CO(GT)'].isna()]
X = df.drop(['CO(GT)','NMHC(GT)','C6H6(GT)','NOx(GT)','NO2(GT)'], axis=1).copy()
y = df['CO(GT)']

filter_list_pearson_spearman = list_pearson_spearman.loc[list_pearson_spearman['columna'].isin(X.columns)]
filter_list_pearson_spearman = filter_list_pearson_spearman.sort_values(by= ['valor pearson','valor spearman'], ascending=False)

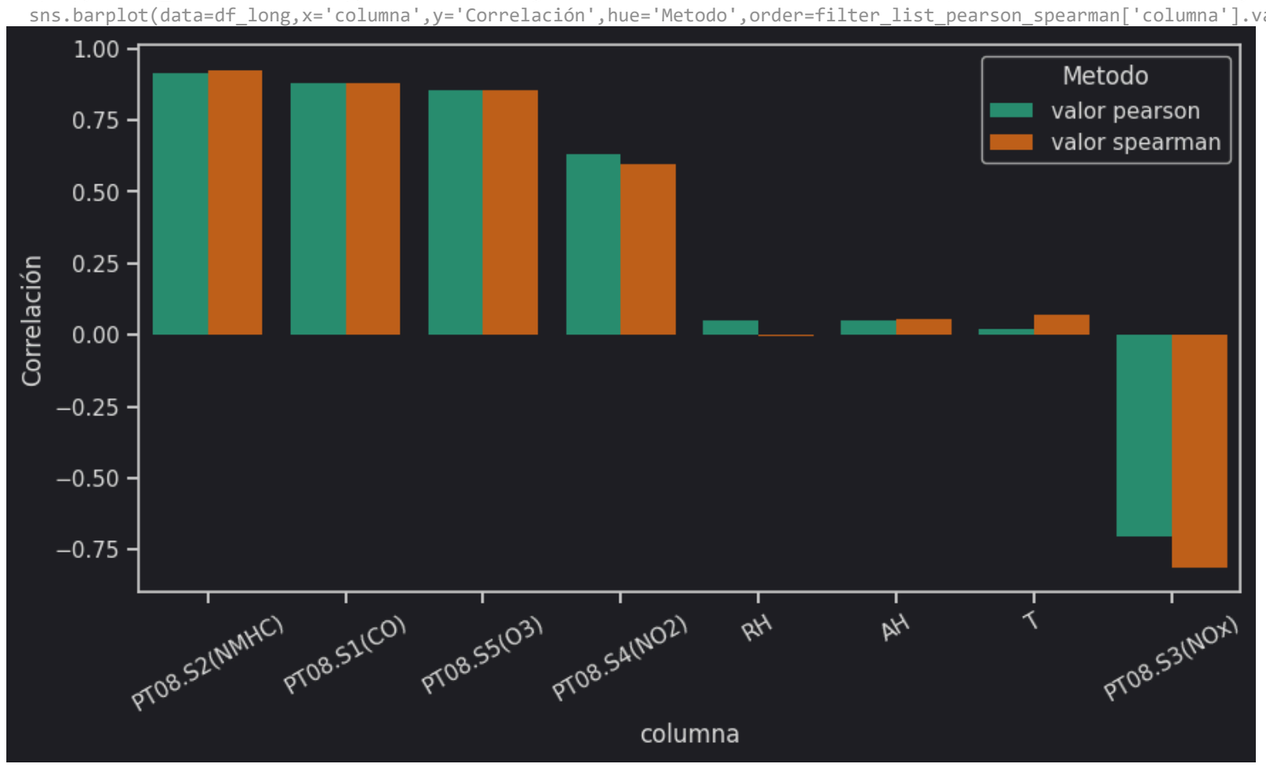
display(filter_list_pearson_spearman)
plt.figure(figsize=(10,5))

df_long = filter_list_pearson_spearman.melt(
    id_vars='columna',
    value_vars=['valor pearson', 'valor spearman'],
    var_name='Metodo',
    value_name='Correlación'
)

sns.barplot(data=df_long,x='columna',y='Correlación',hue='Metodo',order=filter_list_pearson_spearman['columna'].values, palette=sns.color_palette(palette='Dark2'))
plt.xticks(rotation=30)
plt.show()
```

	columna	valor pearson	valor spearman
3	PT08.S2(NMHC)	0.915514	0.926978
0	PT08.S1(CO)	0.879288	0.879422
8	PT08.S5(O3)	0.854182	0.855705
7	PT08.S4(NO2)	0.630703	0.594367
10	RH	0.048890	-0.006484
11	AH	0.048556	0.053475
9	T	0.022109	0.068907
5	PT08.S3(NOx)	-0.703446	-0.811156

```
/tmp/ipykernel_1633/3201825679.py:20: UserWarning: The palette list has more values (8) than needed (2), which may not be intended.
  sns.barplot(data=df_long,x='columna',y='Correlación',hue='Metodo',order=filter_list_pearson_spearman['columna'].values,palette=sns.color_palette(palette='Dark2'))
```



- Realicé este gráfico para visualizar y ordenar de manera más clara la correlación lineal (Pearson) y la correlación monótona (Spearman) de cada variable con respecto a la variable objetivo, la Concentración Real de Monóxido de Carbono.
- La Lectura del Sensor de Hidrocarburos No Metánicos presenta la mayor correlación con la variable objetivo, siendo predominantemente lineal. Le siguen la Lectura del Sensor de Monóxido de Carbono y la Lectura del Sensor de Ozono, que también muestran correlaciones fuertes tanto lineales como monótonas. Las variables Humedad Absoluta, Humedad Relativa y Temperatura del Aire presentan correlaciones débiles, tanto lineales como monótonas, por lo que individualmente parecen tener una capacidad explicativa limitada sobre la concentración de monóxido de carbono.
- Una de las variables más influyentes es la Lectura del Sensor de Óxidos de Nitrógeno. Si bien presenta una correlación lineal negativa fuerte, su correlación monótona es aproximadamente un 10% mayor en valor absoluto. Esta diferencia, sumada a la forma observada en el gráfico de dispersión, sugiere la existencia de una relación inversa no lineal entre ambas variables. Visualmente, dicho comportamiento es compatible con una relación logarítmica negativa, donde incrementos iniciales en la lectura del sensor se asocian con disminuciones pronunciadas en la concentración de monóxido de carbono, mientras que para valores más elevados la tasa de disminución se vuelve progresivamente menor.

Haz doble clic (o ingresa) para editar

```
display((X.isna().sum()))
display(((X.isna().sum() / X.shape[0]) * 100).round(2))
display(X.loc[X['PT08.S2(NMHC)'].isna()].head(5))
```

	0
PT08.S1(CO)	366
PT08.S2(NMHC)	366
PT08.S3(NOx)	366
PT08.S4(NO2)	366
PT08.S5(O3)	366
T	366
RH	366
AH	366

dtype: int64

	0
PT08.S1(CO)	3.91
PT08.S2(NMHC)	3.91
PT08.S3(NOx)	3.91
PT08.S4(NO2)	3.91
PT08.S5(O3)	3.91
T	3.91
RH	3.91
AH	3.91

dtype: float64

	PT08.S1(CO)	PT08.S2(NMHC)	PT08.S3(NOx)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
524	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
525	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
526	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
701	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
702	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Inicialmente consideré utilizar **KNN Imputer** para imputar los valores faltantes presentes en las variables predictoras. Sin embargo, al analizar la distribución de los datos faltantes observé que estos se encuentran concentrados en los mismos registros para todas las variables de entrada. Este comportamiento me lleva a pensar que el sistema de adquisición o almacenamiento de datos sufrió interrupciones completas durante determinados períodos de tiempo, afectando simultáneamente a todos los sensores. En total, estos registros representan aproximadamente el **3.91% del conjunto de datos**, equivalente a unas **366 horas de medición**.
- Debido a que las observaciones afectadas presentan valores faltantes en todas las variables predictoras, considero que la aplicación de **KNN Imputer** no es la alternativa más adecuada. Aunque el algoritmo podría generar imputaciones utilizando otras observaciones del conjunto de datos, la ausencia total de información en estos registros reduce la confiabilidad de las estimaciones y podría introducir ruido en el modelo.
- Por este motivo, y considerando que el porcentaje de registros afectados es relativamente bajo, decidí eliminar estas observaciones del conjunto de datos. En consecuencia, también eliminaré los valores correspondientes de la variable objetivo (**CO(GT)**), ya que no dispongo de información suficiente en las variables predictoras para estimar de manera fiable dichas observaciones.

```
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.model_selection import train_test_split



# Filtrar filas donde 'CO(GT)' o 'PT08.S2(NMHC)' son NaN, y luego eliminar columnas innecesarias
X_filtered = df.loc[~(df['PT08.S2(NMHC)'].isna()) & ~(df['CO(GT)'].isna())].copy()
y = X_filtered['CO(GT)']
X = X_filtered.drop(['CO(GT)','NMHC(GT)','C6H6(GT)','NOx(GT)','NO2(GT)'], axis=1)
X_columns = X.columns
# transformo la variable 'PT08.S3(NOx)' a logaritmo natural
figure, axs = plt.subplots(1,2,figsize=(15,5))

copy_PT08 = X['PT08.S3(NOx)'].copy()
sns.scatterplot(x=copy_PT08, y=y,ax=axs[0])

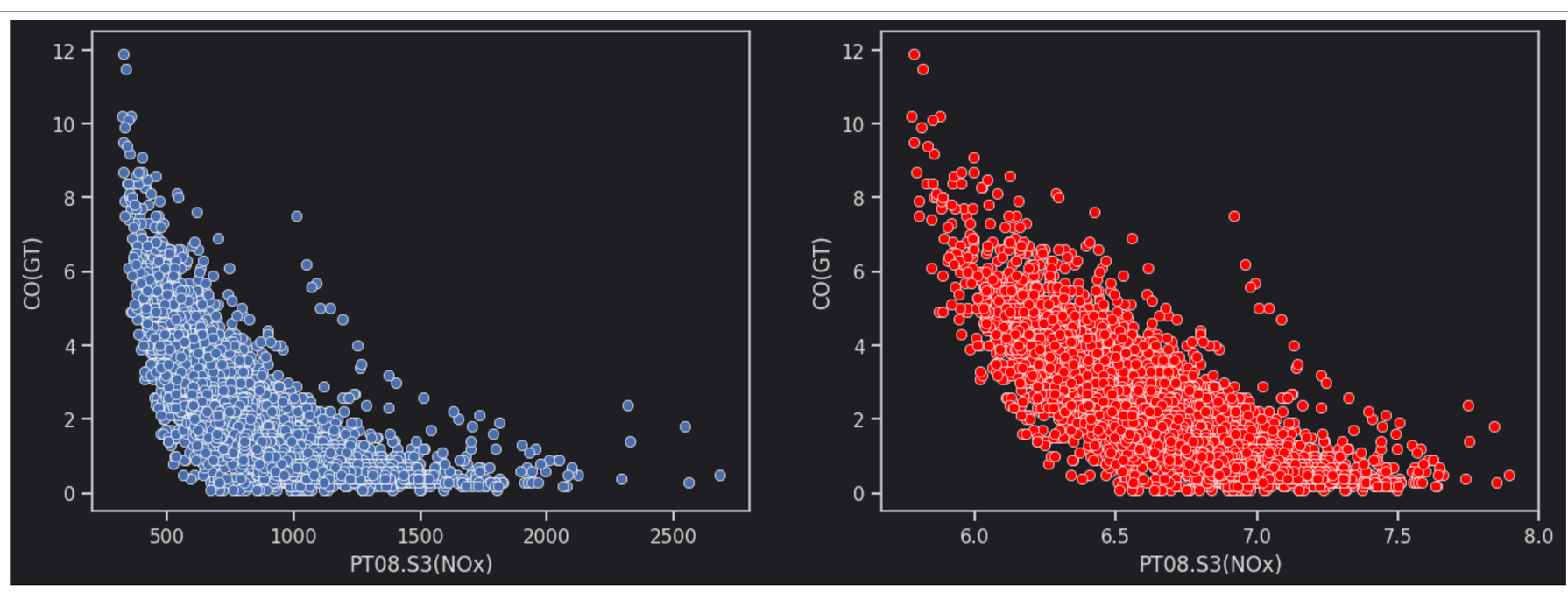
X['PT08.S3(NOx)'] = np.log(X['PT08.S3(NOx)'])

sns.scatterplot(x=X['PT08.S3(NOx)'], y=y,ax=axs[1],color='red')
plt.show()

X_df = pd.DataFrame(X,columns=X_columns)
# luego de limpiar y transformar los datos, procedo a escalar todas las variables a una misma unidad para que los predictores no se vean afectados por las diferentes escalas de las medicion
standar = StandardScaler()
X_linear = standar.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_linear, y, test_size=0.3, random_state=42)

```



- Se suele utilizar el logaritmo natural para transformar variables que presentan relaciones no lineales, ya que esta transformación puede ayudar a linealizar la relación entre las variables y facilitar su modelado mediante técnicas lineales. En particular, cuando una

relación presenta una forma compatible con una curva logarítmica, la aplicación del logaritmo natural al predictor puede mejorar la representación de la correlación y aumentar la capacidad explicativa de modelos como la regresión lineal.

- Luego de limpiar y transformar los datos, procedo a escalar todas las variables predictoras a una misma escala. El objetivo de este paso es evitar que las diferencias en las unidades de medida o en los rangos de valores influyan de manera desproporcionada en el entrenamiento del modelo.
- De esta forma, todas las variables contribuyen en condiciones comparables y los algoritmos que se basan en distancias o en la magnitud de los coeficientes pueden evaluar los patrones presentes en los datos sin verse sesgados por la escala original de las mediciones.
- por ultimo separo los conjuntos de entrenamiento y testeo en el que utilizo la distribución 70/30, 70% para entrenar 30% para testear

Haz doble clic (o ingresa) para editar

```
1 display(((X_df.isna().sum() / X_df.shape[0]) * 100).round(2))
```

	0
PT08.S1(CO)	0.0
PT08.S2(NMHC)	0.0
PT08.S3(NOx)	0.0
PT08.S4(NO2)	0.0
PT08.S5(O3)	0.0
T	0.0
RH	0.0
AH	0.0

dtype: float64

verifico que no tenga nulos

Resolución:

```
1 LinearRegression = LinearRegression()
2 modelo = LinearRegression.fit(X_train,y_train)
3 modelo
```

▾ LinearRegression

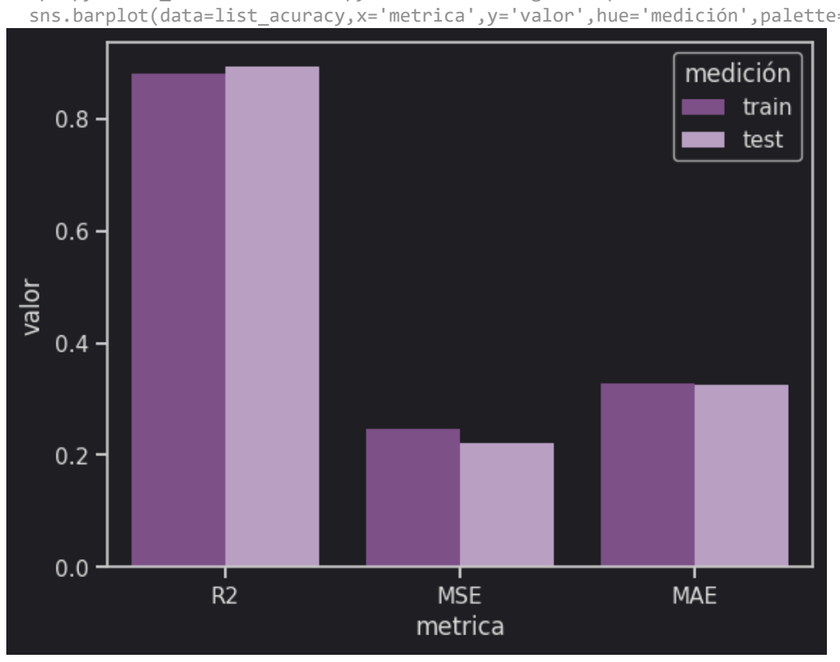
```
LinearRegression()
```

4. **Regresión Lineal**

- Ajusten un modelo de regresión lineal (usando `sklearn.linear_model.LinearRegression`).
- Obtengan:
  - Coeficientes (pesos) del modelo.
  - Evaluar métricas en el set de entrenamiento y en el de testeo: **R^2**, MSE (error cuadrático medio), MAE.

```
1 #r2_score, mean_squared_error, mean_absolute_error
2 prediccon_train = modelo.predict(X_train)
3 prediccon_test = modelo.predict(X_test)
4
5 list_acuracy = [
6     {'medición' : 'train','R2' : r2_score(y_train,prediccon_train),'MSE' : mean_squared_error(y_train,prediccon_train),'MAE' : mean_absolute_error(y_train,prediccon_train)},
7     {'medición' : 'test','R2' : r2_score(y_test,prediccon_test),'MSE' : mean_squared_error(y_test,prediccon_test),'MAE' : mean_absolute_error(y_test,prediccon_test)}
8 ]
9
10 list_acuracy = pd.DataFrame(list_acuracy)
11 list_acuracy = list_acuracy.melt(id_vars='medición',var_name='metrica',value_name='valor')
12
13 sns.barplot(data=list_acuracy,x='metrica',y='valor',hue='medición',palette=sns.color_palette(palette='PRGn'))
14 plt.show()
```

```
/tmp/ipykernel_1633/1968187677.py:13: UserWarning: The palette list has more values (6) than needed (2), which may not be intended.
  sns.barplot(data=list_acuracy,x='metrica',y='valor',hue='medición',palette=sns.color_palette(palette='PRGn'))
```



- Si bien cada métrica evalúa el desempeño del modelo desde una perspectiva diferente, el gráfico permite observar que no existe evidencia significativa de sobreajuste (overfitting). Los resultados obtenidos en entrenamiento y prueba son muy similares tanto para R^2 como para MAE, lo que indica una buena capacidad de generalización del modelo.
- La única diferencia apreciable se observa en la métrica MSE (Error Cuadrático Medio), donde el desempeño en entrenamiento es ligeramente mejor que en entrenamiento. Esto podría sugerir un leve sobreajuste. Sin embargo, dado que el MSE penaliza más severamente los errores grandes que el MAE, esta diferencia podría estar asociada a la presencia de algunas observaciones con errores elevados en el conjunto de prueba más que a un problema real de generalización del modelo.

```
1 list_acuracy.loc[list_acuracy['medición'] =='test']
2 list_acuracy.loc[list_acuracy['medición'] =='test']
3 test_metrica = list_acuracy[list_acuracy['medición'] =='test']
4 root_metrica = np.sqrt(test_metrica.loc[test_metrica['metrica'] == 'MSE','valor'].values)
5 display('R2',test_metrica.loc[test_metrica['metrica'] =='R2','valor'].values)
6 display('SMSE',root_metrica)
7 display('MSE',test_metrica.loc[test_metrica['metrica'] =='MSE','valor'].values)
```

```
8 display('MAE',test_metrica.loc[test_metrica['metrica'] =='MAE','valor'].values)
```

```
'R2'
array([0.89309309])
'SMSE'
array([0.47074258])
'MSE'
array([0.22159858])
'MAE'
array([0.32661223])
```

Pasando al conjunto de datos limpio:

- El modelo presenta un **R^2 de 0.8930**, lo que indica que es capaz de explicar aproximadamente el **89.30% de la variabilidad** observada en la concentración real de monóxido de carbono.
- El modelo presenta un **Error Cuadrático Medio (MSE) de 0.22** y una **Raíz del Error Cuadrático Medio (RMSE) de 0.47**. Esto significa que, en promedio, las predicciones del modelo se desvían aproximadamente **0.47 unidades** respecto de los valores reales de la variable objetivo.
- Según la métrica de **Error Absoluto Medio (MAE)**, el modelo se equivoca en promedio **0.32 unidades** respecto del valor real.
- La diferencia entre el **MAE (0.32)** y el **RMSE (0.47)** se debe a que el RMSE penaliza con mayor intensidad los errores grandes o extremos. Por este motivo, cuando existen algunas predicciones con errores elevados, el RMSE tiende a aumentar más que el MAE. En este caso, la diferencia entre ambas métricas sugiere la presencia de algunos errores relativamente grandes, aunque no lo suficientemente importantes como para afectar significativamente el desempeño general del modelo.

5. Importancia de Variables

- Analicen los **coeficientes** del modelo:
  - Valores absolutos altos → mayor impacto en el target.
  - Signo: Relación positiva/negativa con el target.
- Comparen la magnitud de los coeficientes **escalados** (si usaron features en distintas unidades).
- **5.1. Opcional**: Otra forma es "desordenar" un feature y ver cómo empeora el modelo. Si al desordenarlo el error aumenta mucho, ese feature era importante. Usen métodos como:
  - **Permutation Importance** (de sklearn) para validar importancia. Más info en https://scikit-learn.org/stable/modules/permutation_importance.html

```
lista_coeficientes_abs = np.abs(modelo.coef_)
lista_coeficientes = modelo.coef_
fig_coef,ax_coef = plt.subplots(1,2,figsize=(10,5))

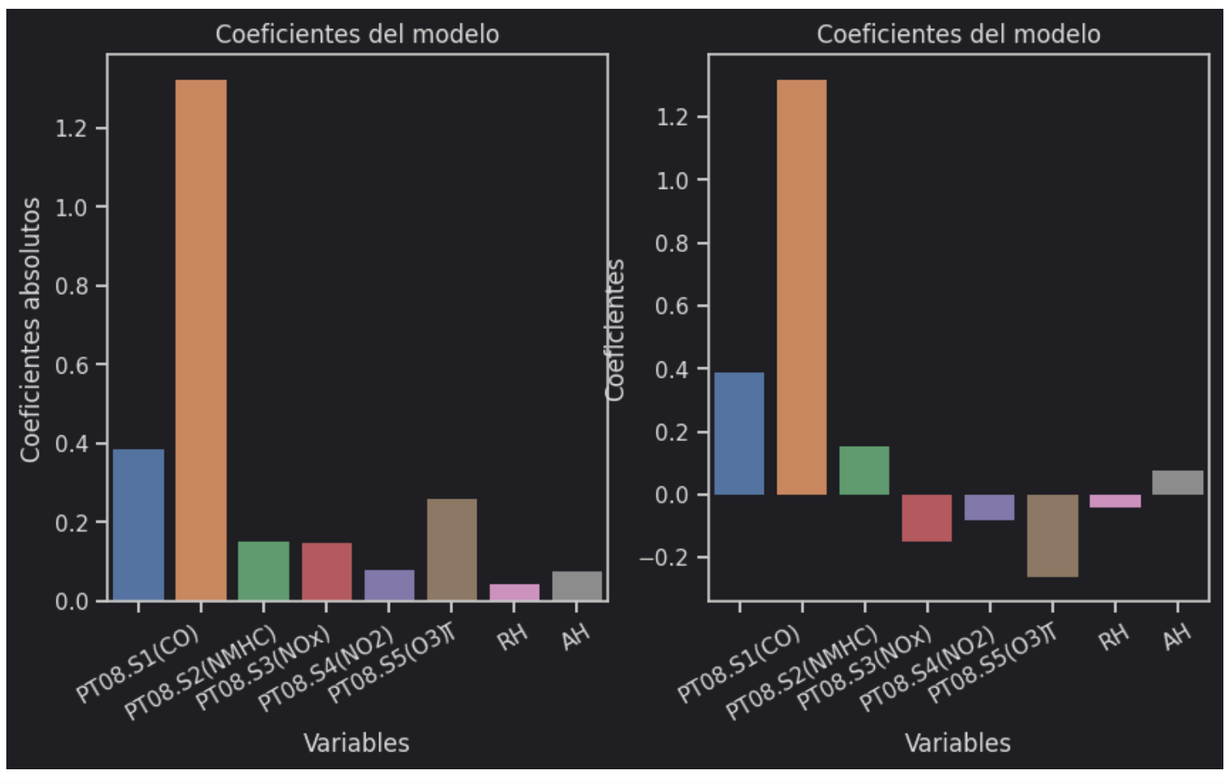
sns.barplot(x=X_columns,y=lista_coeficientes_abs,hue=X_columns,ax=ax_coef[0])
ax_coef[0].set_title('Coeficientes del modelo')
ax_coef[0].set_xlabel('Variables')
ax_coef[0].set_ylabel('Coeficientes absolutos')
ax_coef[0].tick_params(axis='x',rotation=30)
ax_coef[1].tick_params(axis='x',rotation=30)

sns.barplot(x=X_columns,y=lista_coeficientes,hue=X_columns,ax=ax_coef[1])
ax_coef[1].set_title('Coeficientes del modelo')
ax_coef[1].set_xlabel('Variables')
ax_coef[1].set_ylabel('Coeficientes')

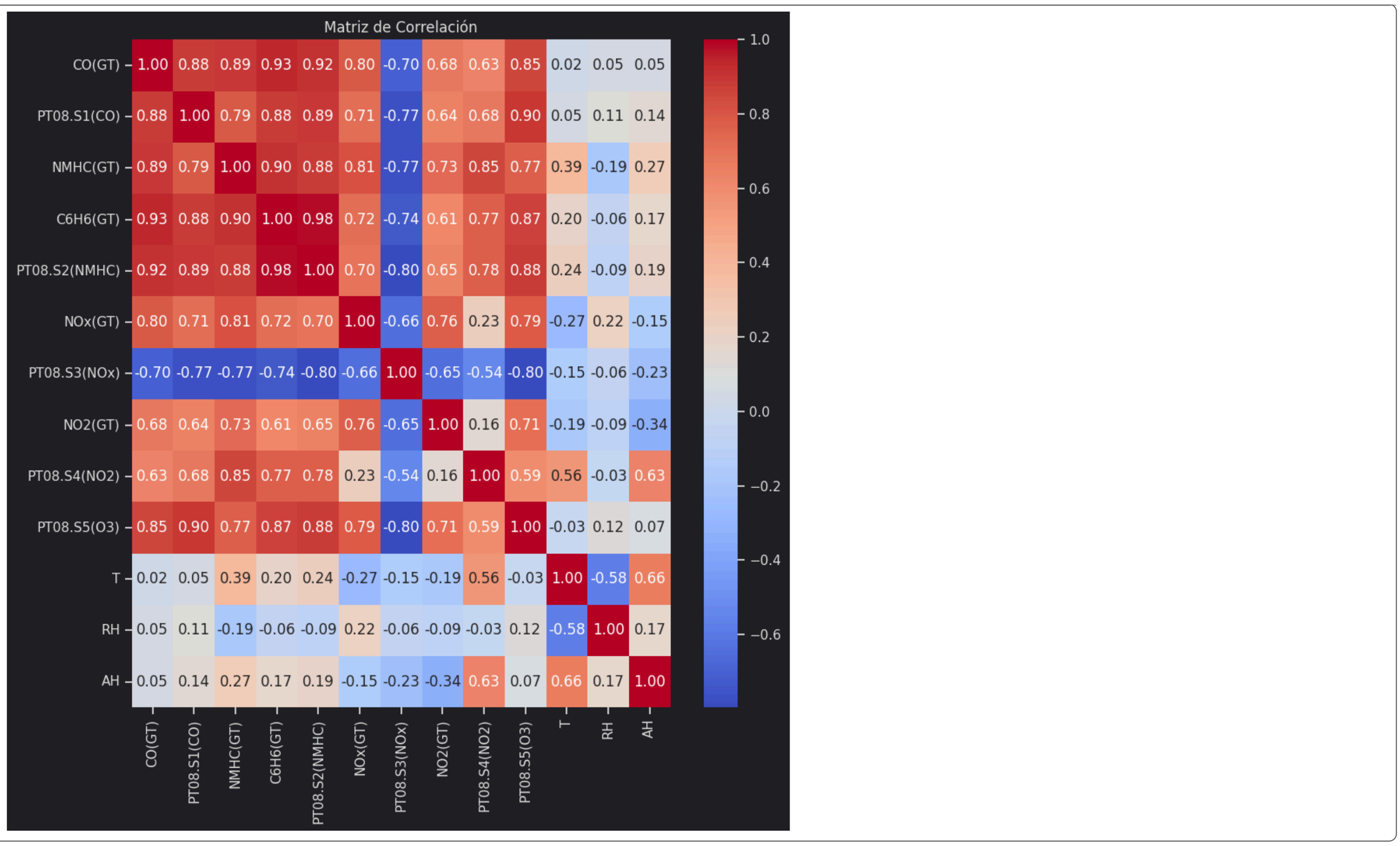

plt.show()

#df_coeficientes = pd.DataFrame(lista_coeficientes,columns=lista_nombres_coeficientes)

```




```
df_corr = df.corr()
plt.figure(figsize=(10,10))
fig_corr = sns.heatmap(df_corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Matriz de Correlación')
plt.show()
```



- Observando la magnitud de los coeficientes, puedo notar que el modelo le asigna una importancia mucho mayor a la **Lectura del Sensor de Hidrocarburos No Metánicos** que al propio **Sensor de Monóxido de Carbono** para predecir la concentración real de monóxido de carbono.
- Una posible explicación es que, en las ciudades, una de las principales fuentes de emisión de monóxido de carbono son los vehículos. Estos también emiten hidrocarburos no metánicos, por lo que ambas sustancias suelen aumentar y disminuir al mismo tiempo. Debido a esta relación, la medición de hidrocarburos resulta ser un muy buen indicador de la presencia de monóxido de carbono.
- Esta diferencia puede deberse a que el sensor de hidrocarburos tenga menos ruido o una relación más estable con la concentración real de monóxido de carbono, lo que lleva al modelo a confiar más en esta variable al momento de realizar las predicciones.
- El modelo utiliza el sensor de hidrocarburos como predictor principal y utiliza el resto de predictores como respaldo para mayor precisión.
- Tanto en el analisis previo como en los resultados del modelo se puede verificar que los predictores de humedad y temperaturas no son relevantes para predecir la variable objetivo

Importancia de las características por permutación

```
from sklearn.inspection import permutation_importance
resultado = permutation_importance(modelo, X_test, y_test, scoring='neg_mean_squared_error', n_repeats=10, random_state=42)

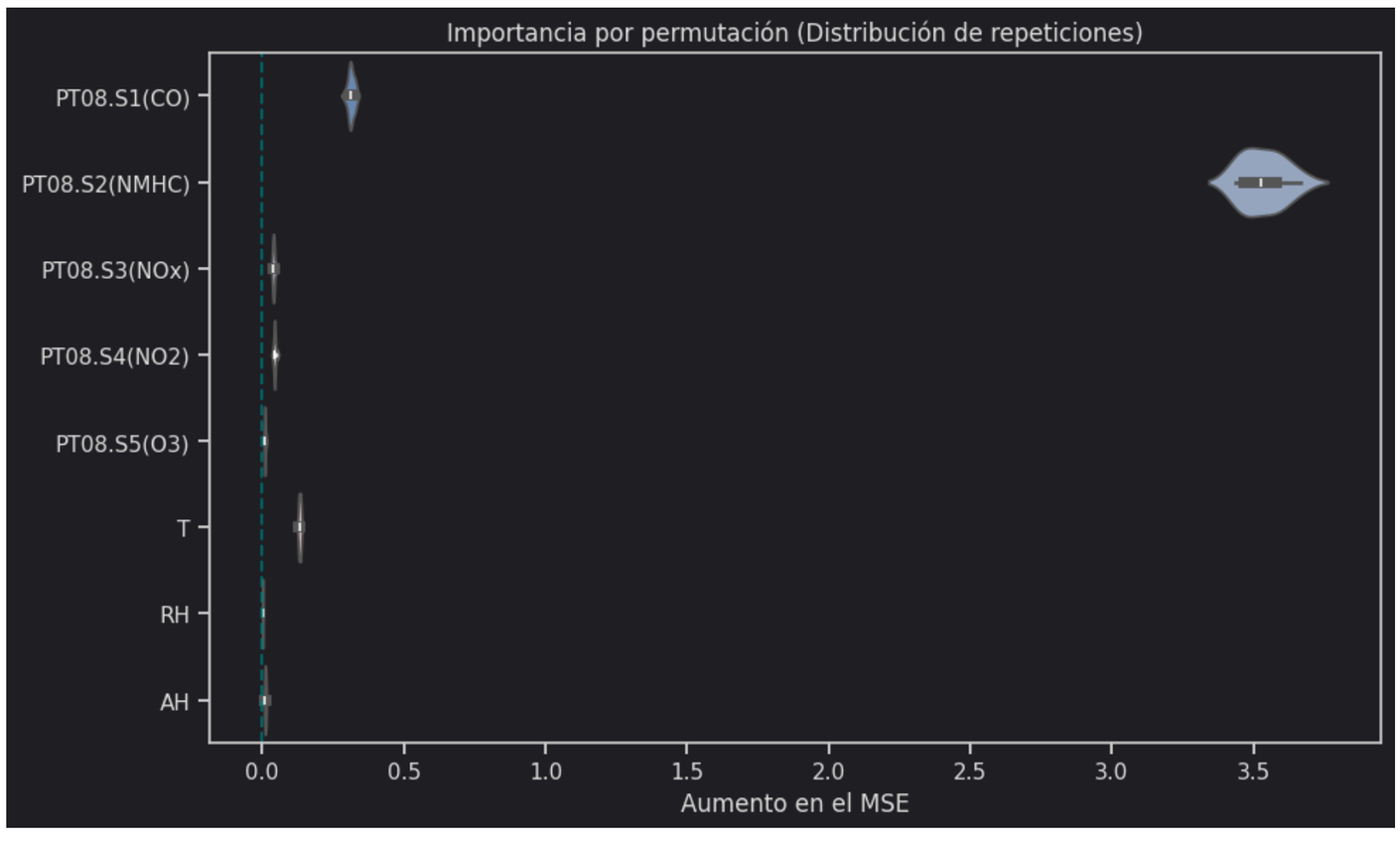
# 1. Creamos un DataFrame con las 10 repeticiones por cada columna
df_importancias = pd.DataFrame(resultado.importances.T, columns=X_columns)

# 2. Graficamos usando el DataFrame completo
plt.figure(figsize=(10, 6))
sns.violinplot(data=df_importancias, orient="h", inner="box", palette="vlag")

plt.xlabel("Aumento en el MSE")
plt.title("Importancia por permutación (Distribución de repeticiones)")
plt.axvline(x=0, color="teal", linestyle="--", linewidth=1.5, alpha=0.7)


plt.tight_layout()
plt.show()

```



- En este gráfico se visualiza cuánto aumenta el error cuadrático medio (MSE) cuando se altera aleatoriamente cada predictor. Cuanto mayor sea el aumento del error, mayor será la dependencia que tiene el modelo de esa variable para realizar sus predicciones.
- Se puede observar que la Lectura del Sensor de Hidrocarburos No Metánicos es la variable más importante para el modelo, ya que al invalidarla se produce el mayor aumento en el error. Muy por detrás aparece la Lectura del Sensor de Monóxido de Carbono, lo que refuerza la idea de que el modelo se apoya principalmente en la información proporcionada por el sensor de hidrocarburos.
- A diferencia del análisis basado en correlaciones y coeficientes, en este caso la Temperatura del Aire muestra una importancia considerable dentro del modelo. Esto puede deberse a que, aunque por sí sola presenta una correlación muy baja con la variable objetivo, aporta información útil cuando se combina con el resto de los predictores. Este comportamiento es habitual en modelos multivariados, donde una variable puede tener poca relación individual con el objetivo pero contribuir a mejorar las predicciones al interactuar con otras variables.

Haz doble clic (o ingresa) para editar

6. **Reflexión**

- ¿Cuáles variables son más importantes según el modelo? ¿Coincide con su análisis exploratorio?
- ¿El modelo tiene buen rendimiento (R^2 alto, MSE bajo)? Si no, ¿a qué podría deberse?

Resolución:

Bonus:

- Prueben eliminar variables "poco importantes" y reentrenar el modelo. ¿Mejora el rendimiento?

Pruevo con quitar las variables correspondientes a la humedad relativa y humedad absoluta

```
from sklearn.linear_model import LinearRegression
X_2 = X_df[['PT08.S1(CO)', 'PT08.S2(NMHC)', 'PT08.S3(NOx)', 'PT08.S4(NO2)', 'PT08.S5(O3)','T']]
display(X_2.head())

X_columns2 = ['PT08.S1(CO)', 'PT08.S2(NMHC)', 'PT08.S3(NOx)', 'PT08.S4(NO2)', 'PT08.S5(O3)','T']

X_train2, X_test2, y_train2, y_test2 = train_test_split(X_2, y, test_size=0.3, random_state=42)

modelo2 = LinearRegression()
modelo2 = modelo2.fit(X_train2,y_train2)

prediccon_train2 = modelo2.predict(X_train2)
prediccon_test2 = modelo2.predict(X_test2)
```

	PT08.S1(CO)	PT08.S2(NMHC)	PT08.S3(NOx)	PT08.S4(NO2)	PT08.S5(O3)	T
0	1360.0	1046.0	6.962243	1692.0	1268.0	13.6
1	1292.0	955.0	7.068172	1559.0	972.0	13.3
2	1402.0	939.0	7.038784	1555.0	1074.0	11.9
3	1376.0	948.0	6.995766	1584.0	1203.0	11.0
4	1272.0	836.0	7.094235	1490.0	1110.0	11.2

```
import seaborn as sns
#r2_score, mean_squared_error, mean_absolute_error
list_acuracy = [
    {'medición' : 'train','R2' : r2_score(y_train2,prediccon_train2),'MSE' : mean_squared_error(y_train,prediccon_train),'MAE' : mean_absolute_error(y_train2,prediccon_train2)},
    {'medición' : 'test','R2' : r2_score(y_test2,prediccon_test2),'MSE' : mean_squared_error(y_test2,prediccon_test2),'MAE' : mean_absolute_error(y_test2,prediccon_test2)}
]

list_acuracy = pd.DataFrame(list_acuracy)
list_acuracy = list_acuracy.melt(id_vars='medición',var_name='metrica',value_name='valor')

sns.barplot(data=list_acuracy,x='metrica',y='valor',hue='medición',palette=sns.color_palette(palette='PRGn'))


lista_coeficientes_abs2 = np.abs(modelo2.coef_)
lista_coeficientes2 = modelo2.coef_
fig_coef2,ax_coef2 = plt.subplots(1,2,figsize=(10,5))

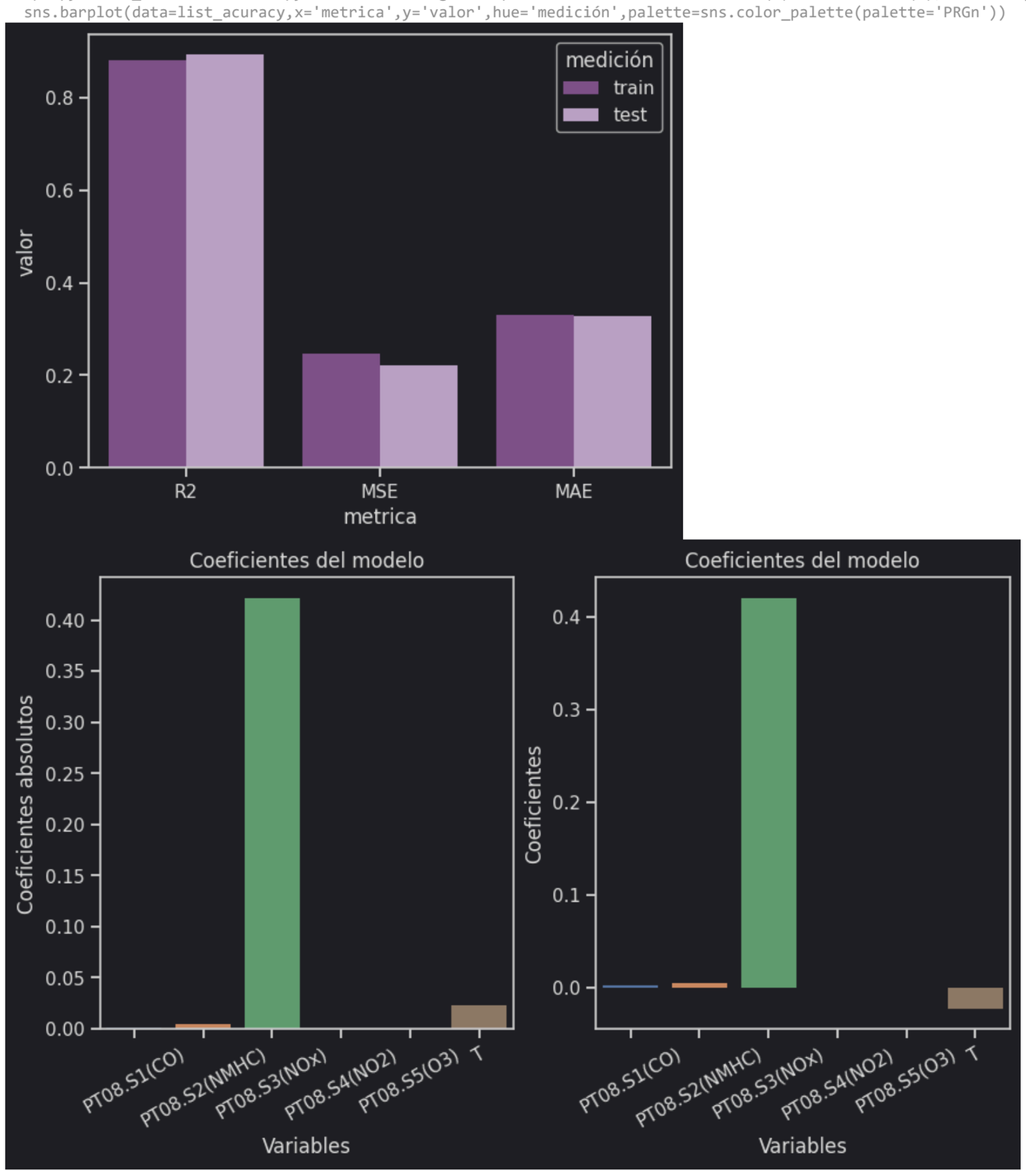
sns.barplot(x=X_columns2,y=lista_coeficientes_abs2,hue=X_columns2,ax=ax_coef2[0])
ax_coef2[0].set_title('Coeficientes del modelo')
ax_coef2[0].set_xlabel('Variables')
ax_coef2[0].set_ylabel('Coeficientes absolutos')
ax_coef2[0].tick_params(axis='x',rotation=30)
ax_coef2[1].tick_params(axis='x',rotation=30)

sns.barplot(x=X_columns2,y=lista_coeficientes2,hue=X_columns2,ax=ax_coef2[1])
ax_coef2[1].set_title('Coeficientes del modelo')
ax_coef2[1].set_xlabel('Variables')
ax_coef2[1].set_ylabel('Coeficientes')



plt.show()
```

```
/tmp/ipykernel_1633/2529449692.py:11: UserWarning: The palette list has more values (6) than needed (2), which may not be intended.
  sns.barplot(data=list_acuracy,x='metrica',y='valor',hue='medición',palette=sns.color_palette(palette='PRGn'))
```




```
1 df
```

	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
0	2.6	1360.0	150.0	11.9	1046.0	166.0	1056.0	113.0	1692.0	1268.0	13.6	48.9	0.7578
1	2.0	1292.0	112.0	9.4	955.0	103.0	1174.0	92.0	1559.0	972.0	13.3	47.7	0.7255
2	2.2	1402.0	88.0	9.0	939.0	131.0	1140.0	114.0	1555.0	1074.0	11.9	54.0	0.7502
3	2.2	1376.0	80.0	9.2	948.0	172.0	1092.0	122.0	1584.0	1203.0	11.0	60.0	0.7867
4	1.6	1272.0	51.0	6.5	836.0	131.0	1205.0	116.0	1490.0	1110.0	11.2	59.6	0.7888
...	...	...	...	...	...	...	...	...	...	...	...	...	...
9352	3.1	1314.0	NaN	13.5	1101.0	472.0	539.0	190.0	1374.0	1729.0	21.9	29.3	0.7568
9353	2.4	1163.0	NaN	11.4	1027.0	353.0	604.0	179.0	1264.0	1269.0	24.3	23.7	0.7119
9354	2.4	1142.0	NaN	12.4	1063.0	293.0	603.0	175.0	1241.0	1092.0	26.9	18.3	0.6406
9355	2.1	1003.0	NaN	9.5	961.0	235.0	702.0	156.0	1041.0	770.0	28.3	13.5	0.5139
9356	2.2	1071.0	NaN	11.9	1047.0	265.0	654.0	168.0	1129.0	816.0	28.5	13.1	0.5028

9357 rows × 13 columns

```
1  X = X[X_columns2]
2  X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
3
4  """import matplotlib.pyplot as plt
5  from sklearn.preprocessing import PowerTransformer
6  from sklearn.metrics import mean_squared_error, r2_score
7  from sklearn.model_selection import GridSearchCV, train_test_split
8  from sklearn.neural_network import MLPRegressor
9
10
11
12
13
14
15
16 box_cox = PowerTransformer(method='yeo-johnson', standardize=True)
17
18
19 X_train_scaled = box_cox.fit_transform(X_train_raw)
20
21
22 X_test_scaled = box_cox.transform(X_test_raw)
23
24
25 mlp_base = MLPRegressor(max_iter=100, early_stopping=True, random_state=42)
26
27 # 4. Definimos la cuadrícula (Grid) de hiperparámetros
28 param_grid = {
29     'hidden_layer_sizes': [(5,), (10,), (10, 5)],
30     'activation': ['relu', 'tanh'],
31     'alpha': [0.0001, 0.01, 0.1],
32     "batch_size" : [50],
33     'learning_rate_init': [0.001, 0.01,0.1]
34 }
35
36 print("Iniciando la búsqueda en cuadrícula. Esto probará múltiples modelos...")
37 print(f"Total de combinaciones a probar: {len(param_grid['hidden_layer_sizes']) * len(param_grid['activation']) * len(param_grid['alpha']) * len(param_grid['learning_rate_init'])}")
38
39 # 5. Configuramos GridSearchCV
40 grid_search = GridSearchCV(
41     estimator=mlp_base,
42     param_grid=param_grid,
43     cv=3,
44     n_jobs=-1,
45     scoring='neg_mean_squared_error',
46     verbose=1
47 )
```

```
# 6. Entrenamos (fit) la búsqueda con los datos escalados de entrenamiento
grid_search.fit(X_train_scaled, y_train)

# 7. Obtenemos la mejor configuración
print("\n--- Resultados de la Búsqueda ---")
print("Mejores hiperparámetros encontrados:")
print(grid_search.best_params_)


mejor_modelo = grid_search.best_estimator_


y_pred = mejor_modelo.predict(X_test_scaled)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"\nRendimiento en el conjunto de prueba (Test Set):")
print(f"Error Cuadrático Medio (MSE): {mse:.4f}")
print(f"Coeficiente de Determinación (R2): {r2:.4f} (Más cerca de 1.0 es mejor)")


plt.figure(figsize=(8, 8))


plt.scatter(y_test, y_pred, color='dodgerblue', alpha=0.6, edgecolor='k', label='Predicciones')


min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())


plt.plot([min_val, max_val], [min_val, max_val], color='red', linestyle='--', lw=2, label='Predicción Perfecta')

plt.title('Rendimiento del Modelo: Valores Reales vs Predicciones', fontsize=14)
plt.xlabel('Valores Reales (y_test)', fontsize=12)
plt.ylabel('Predicciones del Modelo (y_pred)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
"""
```

```
'"import matplotlib.pyplot as plt\nfrom sklearn.preprocessing import PowerTransformer\nfrom sklearn.metrics import mean_squared_error, r2_score\nfrom sklearn.model_selection import GridSearchCV, train_test_split\nfrom sklearn.neural_network import MLPRegressor\n\n\n\n\n\n\nbox_cox = PowerTransformer(method=\'yeo-johnson\', standardize=True)\n\n\nX_train_scaled = box_cox.fit_transform(X_train_raw)\n\n\nX_test_scaled = box_cox.transform(X_test_raw)\n\n\nmlp_base = MLPRegressor(max_iter=100, early_stopping=True, random_state=42)\n\n# 4. Definimos la cuadrícula (Grid) de hiperparámetros\nparam_grid = {\n    \'hidden_layer_sizes\': [(5,), (10,), (10, 5)],\n    \'activation\': [\'relu\', \'tanh\'],\n    \'alpha\': [0.0001, 0.01, 0.1],\n    "batch_size" : [50],\n    \'learning_rate_init\': [0.001, 0.01,0.1]\n}\n\nprint("Iniciando la búsqueda en cuadrícula. Esto probará múltiples modelos...")\nprint(f"Total de combinaciones a probar: {len(param_grid[\'hidden_layer_sizes\']) * len(param_grid[\…'
```

importando pytorch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

advice = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(42)
torch.cuda.manual_seed(42)
torch.cuda.manual_seed_all(42)

```

Convirtiendo los datos de entrenamiento y testeo a tensores y luego a un dataLoader

```
import torch
from torch.utils.data import DataLoader, TensorDataset

# Convert pandas DataFrames/Series to NumPy arrays, then to PyTorch tensors
X_train_tensor = torch.tensor(X_train_raw.values, dtype=torch.float32).to(advice)
X_test_tensor = torch.tensor(X_test_raw.values, dtype=torch.float32).to(advice)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).to(advice)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).to(advice)

dataset_entrenamiento = TensorDataset(X_train_tensor, y_train_tensor)

dataset_entrenamiento = DataLoader(dataset_entrenamiento, batch_size=32, shuffle=True)

print("X_train_tensor shape:", X_train_tensor.shape)
print("X_test_tensor shape:", X_test_tensor.shape)
print("y_train_tensor shape:", y_train_tensor.shape)
print("y_test_tensor shape:", y_test_tensor.shape)
```

```
X_train_tensor shape: torch.Size([5140, 6])
X_test_tensor shape: torch.Size([2204, 6])
y_train_tensor shape: torch.Size([5140])
y_test_tensor shape: torch.Size([2204])
```

Texto de título predeterminado

Mostrar código

```
class MLPR_1(nn.Module) :
    def __init__(self) :
        super(MLPR_1, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(6, 5),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(5, 1),
        ).to(advice)

    def forward(self, x) :
        return self.model(x)

modelo_1 = MLPR_1()

optimizer_1 = optim.Adam(modelo_1.parameters(), lr=0.001)
criterion = nn.MSELoss()


evolucition_loss_1 = []

for epoch in range(100) :
    for x_train,y_train in dataset_entrenamiento :
        modelo_1.train()
        optimizer_1.zero_grad()
        outputs = modelo_1(x_train)
        loss = criterion(outputs.squeeze(), y_train)
```

```
        loss.backward()
        optimizer_1.step()

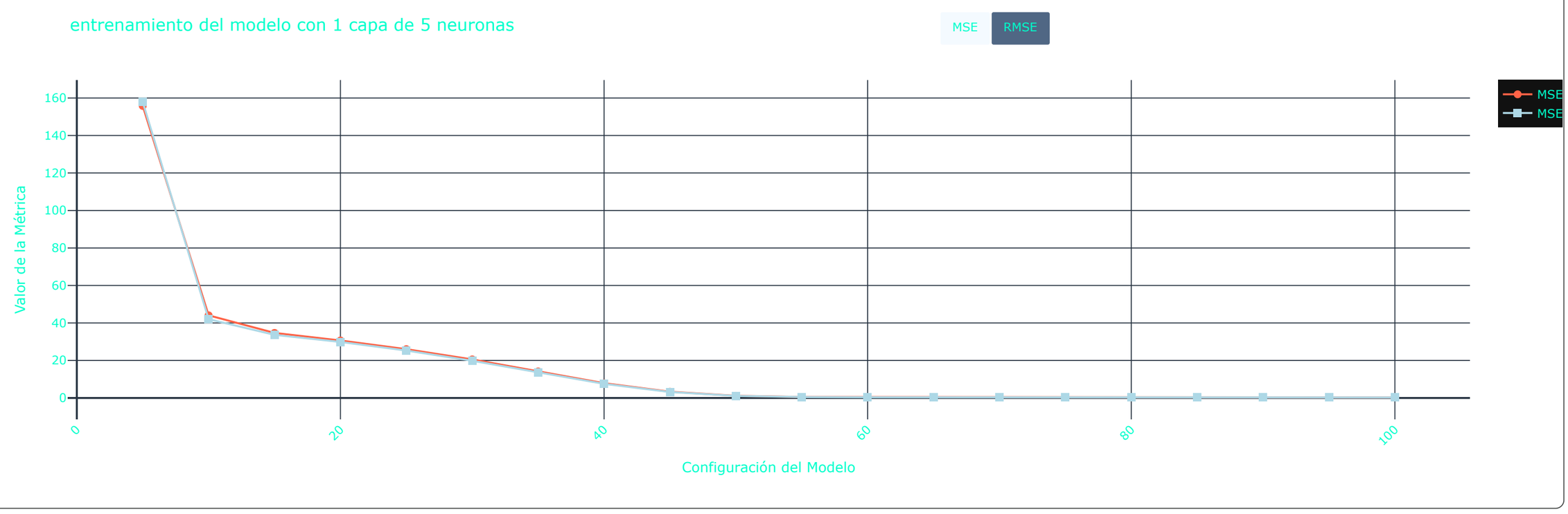
    with torch.no_grad() :
        modelo_1.eval()
        #entrenamiento
        pred_epoch_train = modelo_1(X_train_tensor)
        loss_epoch_train = criterion(pred_epoch_train.squeeze(), y_train_tensor)
        #testeo
        pred_epoch_test = modelo_1(X_test_tensor)
        loss_epoch_test = criterion(pred_epoch_test.squeeze(), y_test_tensor)

    if(epoch+1) % 5 == 0 :
        evolucition_loss_1.append({"epoch": epoch+1 ,
                                   "MSE_train": loss_epoch_train.item(),
                                   "MSE_test": loss_epoch_test.item(),
                                   "RMSE_train": torch.sqrt(loss_epoch_train).item(),
                                   "RMSE_test": torch.sqrt(loss_epoch_test).item()})
```

```
import pandas as pd

evolucition_loss_df = pd.DataFrame(evolucition_loss_1)
evolucition_loss_df

plot_model_performance(evolucition_loss_df,"entrenamiento del modelo con 1 capa de 5 neuronas")
```




```
evolucition_loss_df.loc[evolucition_loss_df["RMSE_test"] == evolucition_loss_df["RMSE_test"].min()]
```

	epoch	MSE_train	MSE_test	RMSE_train	RMSE_test
19	100	0.293288	0.271602	0.541561	0.521154

```
import torch.nn as nn
import torch.optim as optim

class MLPR_2(nn.Module) :
    def __init__(self) :
        super(MLPR_2, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(6, 10),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(10, 1),
        ).to(advice)

    def forward(self, x) :
        return self.model(x)

modelo_2 = MLPR_2()

optimizer_2 = optim.Adam(modelo_2.parameters(), lr=0.001)
criterion = nn.MSELoss()


evolucition_loss_2 = []

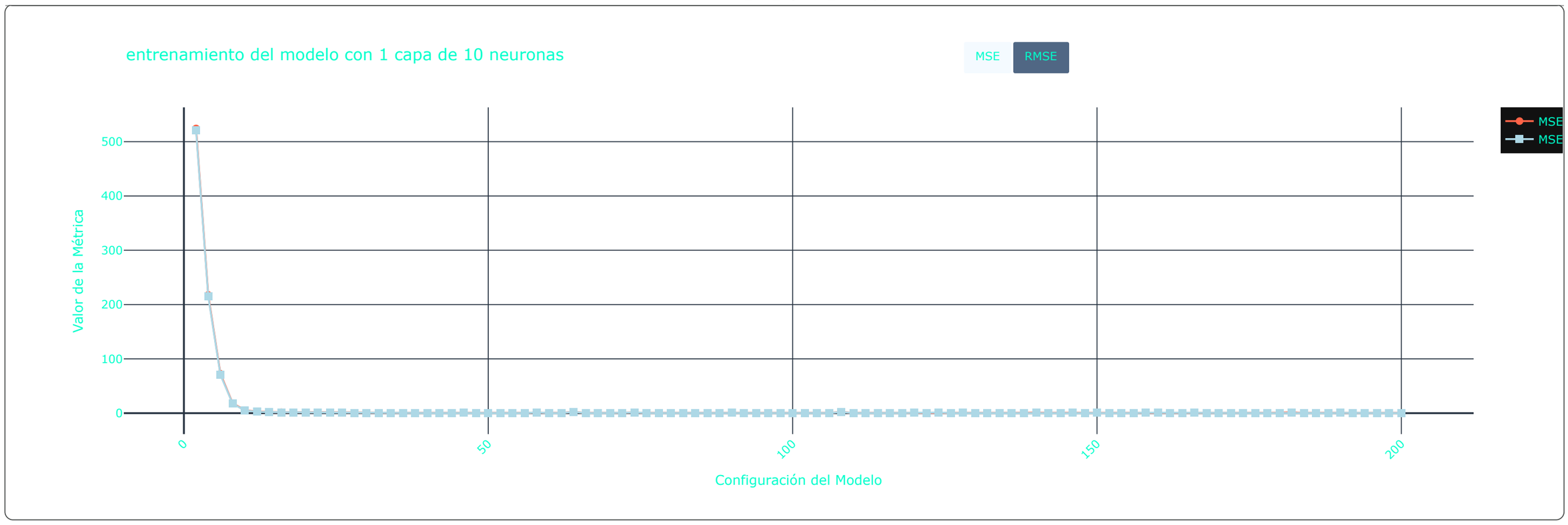
for epoch in range(200) :
    for (x_train,y_train) in dataset_entrenamiento :
        modelo_2.train()
        optimizer_2.zero_grad()
        outputs = modelo_2(x_train)
        loss = criterion(outputs.squeeze(), y_train)
        loss.backward()
        optimizer_2.step()

    with torch.no_grad() :
        modelo_2.eval()
        #entrenamiento
        pred_epoch_train = modelo_2(X_train_tensor)
        loss_epoch_train = criterion(pred_epoch_train.squeeze(), y_train_tensor)
        #testeo
        pred_epoch_test = modelo_2(X_test_tensor)
        loss_epoch_test = criterion(pred_epoch_test.squeeze(), y_test_tensor)

    if(epoch+1) % 2 == 0 :
        evolucition_loss_2.append({"epoch": epoch+1 ,
                                   "MSE_train": loss_epoch_train.item(),
                                   "MSE_test": loss_epoch_test.item(),
                                   "RMSE_train": torch.sqrt(loss_epoch_train).item(),
                                   "RMSE_test": torch.sqrt(loss_epoch_test).item()})
```

```
evolucition_loss_df_2 = pd.DataFrame(evolucition_loss_2)
evolucition_loss_df_2

plot_model_performance(evolucition_loss_df_2,"entrenamiento del modelo con 1 capa de 10 neuronas")
```




```
evolucition_loss_df_2.loc[evolucition_loss_df_2["RMSE_test"] == evolucition_loss_df_2["RMSE_test"].min()]
```

	epoch	MSE_train	MSE_test	RMSE_train	RMSE_test
99	200	0.241953	0.210145	0.491887	0.458416

```
class MLPR_3(nn.Module) :
    def __init__(self) :
        super(MLPR_3, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(6, 15),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(15, 15),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(15, 1),
        ).to(advice)

    def forward(self, x) :
        return self.model(x)

modelo_3 = MLPR_3()

optimizer_3 = optim.Adam(modelo_3.parameters(), lr=0.001)
criterion = nn.MSELoss()


evolucition_loss_3 = []

global_min = float('inf')
checkpoint = {}

for epoch in range(200) :
    for x_train,y_train in dataset_entrenamiento :
        modelo_3.train()
        optimizer_3.zero_grad()
        outputs = modelo_3(x_train)
        loss = criterion(outputs.squeeze(), y_train)
        loss.backward()
        optimizer_3.step()

    with torch.no_grad() :
        modelo_3.eval()
        #entrenamiento
        pred_epoch_train = modelo_3(X_train_tensor)
        loss_epoch_train = criterion(pred_epoch_train.squeeze(), y_train_tensor)
        #testeo
        pred_epoch_test = modelo_3(X_test_tensor)
        loss_epoch_test = criterion(pred_epoch_test.squeeze(), y_test_tensor)

    if global_min > loss_epoch_test :
        global_min = loss_epoch_test

        checkpoint = {
        'epoch': epoch,
        'model_state_dict': modelo_3.state_dict(),
        'optimizer_state_dict': optimizer_3.state_dict()
        # Opcional: puedes guardar también la pérdida (loss) actual
        }

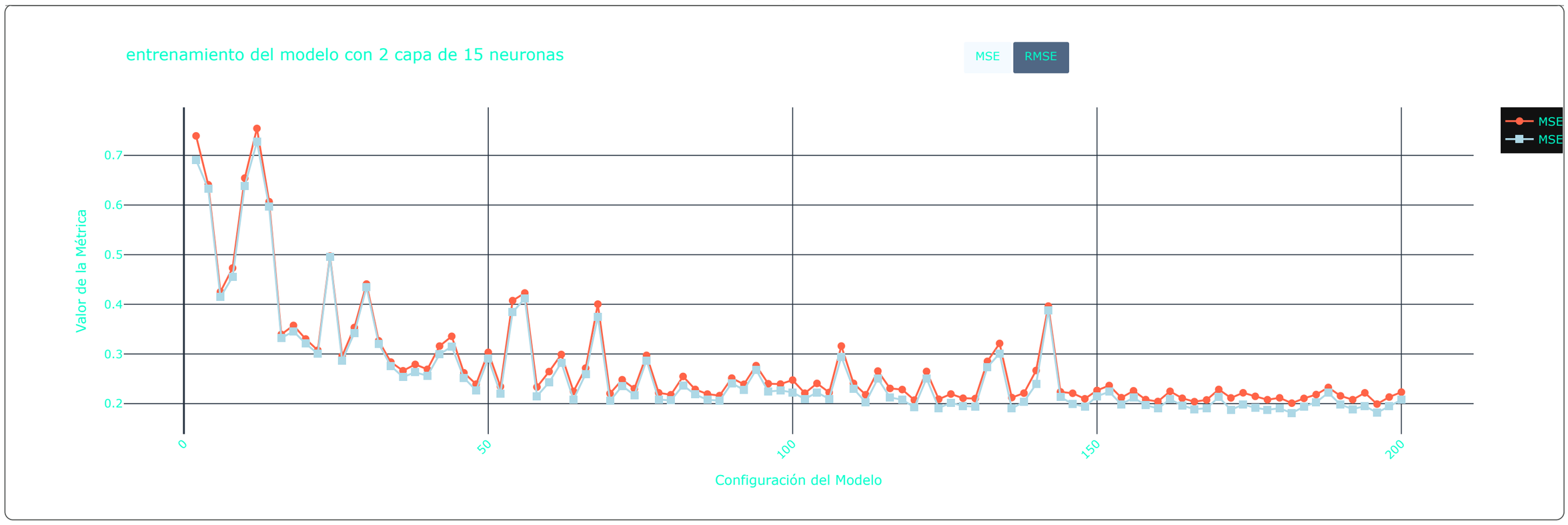


    if(epoch+1) % 2 == 0 :
        evolucition_loss_3.append({"epoch": epoch+1 ,
                                   "MSE_train": loss_epoch_train.item(),
                                   "MSE_test": loss_epoch_test.item(),
                                   "RMSE_train": torch.sqrt(loss_epoch_train).item(),
                                   "RMSE_test": torch.sqrt(loss_epoch_test).item()})

torch.save(checkpoint, f'checkpoint_epoch_{epoch}.pt')
```

```
evolucition_loss_df_3 = pd.DataFrame(evolucition_loss_3)

plot_model_performance(evolucition_loss_df_3,"entrenamiento del modelo con 2 capa de 15 neuronas")
```




```
evolucition_loss_df_3.loc[evolucition_loss_df_3["RMSE_test"] == evolucition_loss_df_3["RMSE_test"].min()]
```

	epoch	MSE_train	MSE_test	RMSE_train	RMSE_test
90	182	0.200989	0.181106	0.448318	0.425566

```
min_rmse_model1 = evolucition_loss_df['RMSE_test'].min()
min_rmse_model2 = evolucition_loss_df_2['RMSE_test'].min()
min_rmse_model3 = evolucition_loss_df_3['RMSE_test'].min()

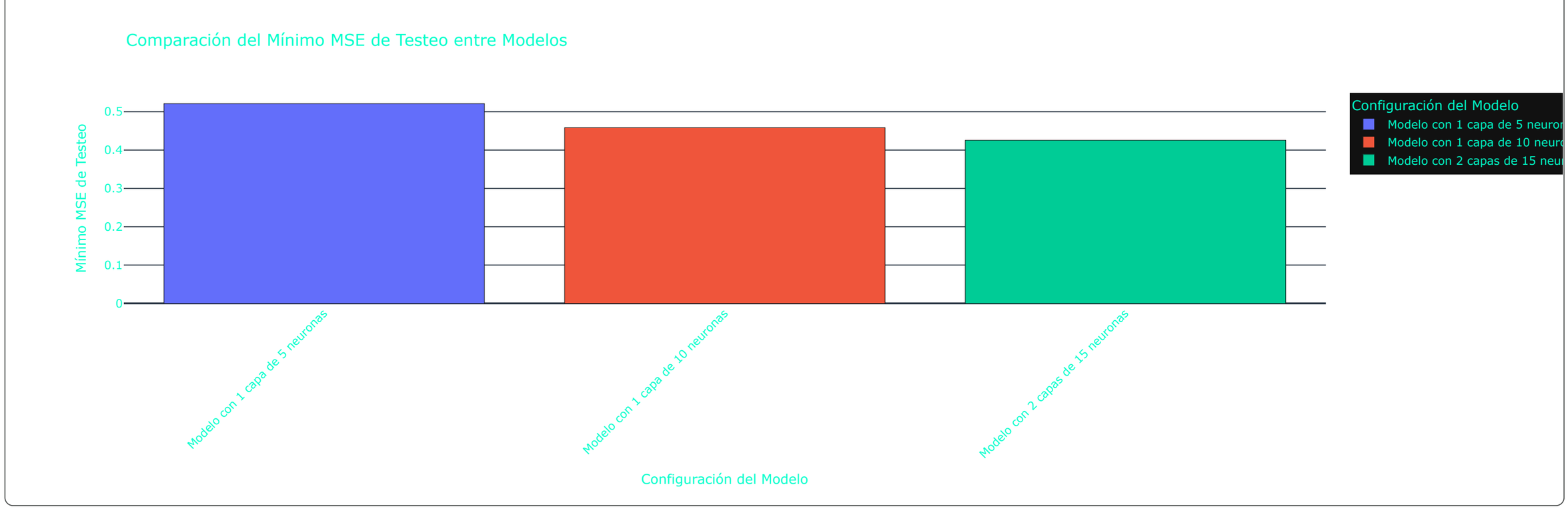
comparison_data = {
    'Modelo': [
        'Modelo con 1 capa de 5 neuronas',
        'Modelo con 1 capa de 10 neuronas',
        'Modelo con 2 capas de 15 neuronas'
    ],
    'Min_MSE_Test': [
        min_rmse_model1,
        min_rmse_model2,
        min_rmse_model3
    ]
}

df_comparison = pd.DataFrame(comparison_data)

fig = px.bar(df_comparison, x='Modelo', y='Min_MSE_Test',
             title='Comparación del Mínimo MSE de Testeo entre Modelos',
             labels={'Min_MSE_Test': 'Mínimo MSE de Testeo', 'Modelo': 'Configuración del Modelo'},
             color='Modelo', # Diferencia las barras por color
             template='plotly_dark') # Usa el tema oscuro

fig.update_layout(
    xaxis_title='Configuración del Modelo',
    yaxis_title='Mínimo MSE de Testeo',
    font=dict(color='#00FFCC'),
    xaxis=dict(tickangle=-45)
)

fig.show()
```




```
class MLPR_4(nn.Module) :
    def __init__(self) :
        super(MLPR_4, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(6, 15),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(15, 25),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(25, 25),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(25, 25),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(25, 25),
            nn.LeakyReLU(negative_slope=0.01),
            nn.Linear(25, 1),
        ).to(advice)

    def forward(self, x) :
        return self.model(x)

modelo_4 = MLPR_4()

optimizer_4 = optim.Adam(modelo_4.parameters(), lr=0.001)
criterion = nn.MSELoss()


evolucition_loss_4 = []
```

```
28
29  for epoch in range(2000) :
30      for x_train,y_train in dataset_entrenamiento :
31          modelo_4.train()
32          optimizer_4.zero_grad()
33          outputs = modelo_4(x_train)
34          loss = criterion(outputs.squeeze(), y_train)
35          loss.backward()
36          optimizer_4.step()
37
38      if(epoch+1) % 20== 0 :
39          with torch.no_grad() :
40              modelo_4.eval()
41              #entrenamiento
42              pred_epoch_train = modelo_4(X_train_tensor)
43              loss_epoch_train = criterion(pred_epoch_train.squeeze(), y_train_tensor)
44              #testeo
45              pred_epoch_test = modelo_4(X_test_tensor)
46              loss_epoch_test = criterion(pred_epoch_test.squeeze(), y_test_tensor)
47              print(f"[{epoch+1}] diferencia : {loss_epoch_test - loss_epoch_train}")
48              evolucition_loss_4.append({"epoch": epoch+1 ,
49                                         "MSE_train": loss_epoch_train.item(),
50                                         "MSE_test": loss_epoch_test.item(),
51                                         "RMSE_train": torch.sqrt(loss_epoch_train).item(),
52                                         "RMSE_test": torch.sqrt(loss_epoch_test).item()})
```

```
[20] diferencia : -0.01405295729637146
[40] diferencia : -0.011918872594833374
[60] diferencia : -0.009258300065994263
[80] diferencia : -0.0172111839056015
[100] diferencia : -0.014043450355529785
[120] diferencia : -0.010297223925590515
[140] diferencia : -0.008189558982849121
[160] diferencia : -0.010305821895599365
[180] diferencia : -0.011290669441223145
[200] diferencia : -0.013594776391983032
[220] diferencia : -0.01240570843219757
[240] diferencia : -0.011797219514846802
[260] diferencia : -0.011873766779899597
[280] diferencia : -0.0139102041721344
[300] diferencia : -0.011539086699485779
[320] diferencia : -0.014227226376533508
[340] diferencia : -0.01268814504146576
[360] diferencia : -0.010607913136482239
[380] diferencia : -0.010793089866638184
[400] diferencia : -0.012605920433998108
[420] diferencia : -0.012694627046585083
[440] diferencia : -0.011035352945327759
[460] diferencia : -0.010483875870704651
[480] diferencia : -0.011082589626312256
[500] diferencia : -0.012150242924690247
[520] diferencia : -0.00782310962677002
[540] diferencia : -0.011938393115997314
[560] diferencia : -0.009498268365859985
[580] diferencia : -0.012308910489082336
[600] diferencia : -0.010010972619056702
[620] diferencia : -0.006097450852394104
[640] diferencia : -0.01191277801990509
[660] diferencia : -0.008333474397659302
[680] diferencia : -0.004504472017288208
[700] diferencia : -0.010633021593093872
[720] diferencia : -0.004751190543174744
[740] diferencia : -0.009504586458206177
[760] diferencia : -0.008407726883888245
[780] diferencia : -0.006941244006156921
[800] diferencia : -0.004991680383682251
[820] diferencia : -0.00997065007686615
[840] diferencia : -0.007730200886726379
[860] diferencia : -0.0035378485918045044
[880] diferencia : -0.003032580018043518
[900] diferencia : -0.0048560649156570435
[920] diferencia : -0.00503087043762207
[940] diferencia : -0.007984697818756104
[960] diferencia : -0.00521501898765564
[980] diferencia : -0.008151695132255554
```